WEEK 6 LAB – API Gateway Security Implementation Report

Prepared by: *Annie Stephen Bassey*
Date: *19/01/2026*
Component: API Gateway Security Implementation

1. Executive Summary

This report documents the implementation of a secure API using AWS API Gateway and AWS Lambda. The project focused on building a serverless REST API and securing it using a custom Lambda authorizer for JWT-style token authentication, alongside essential API security controls including HTTPS enforcement, rate limiting, and access control policies.

Top 3 Security Controls Implemented:

- Custom Lambda Authorizer for token-based authentication and authorization
- API Throttling to prevent abuse and ensure availability
- HTTPS-only access with explicit allow/deny policies based on token validation

2. Scope

In-Scope Components:

- AWS Lambda function (MyAPIweek6) serving as the backend API
- Amazon API Gateway (WEEK6API, later myWEEK6_api)
- Lambda Authorizer (week6Authz) for token validation
- API stage (week6) with throttling settings
- Endpoint: https://ztsesu7i0e.execute-api.us-east-1.amazonaws.com

3. Methodology

The implementation followed a structured, hands-on approach within the AWS Management Console.

Tools & Services Used:

- AWS Lambda
- Amazon API Gateway
- AWS IAM (implicitly via authorizer policies)(cognito)
- Web browser for testing

Process Overview:

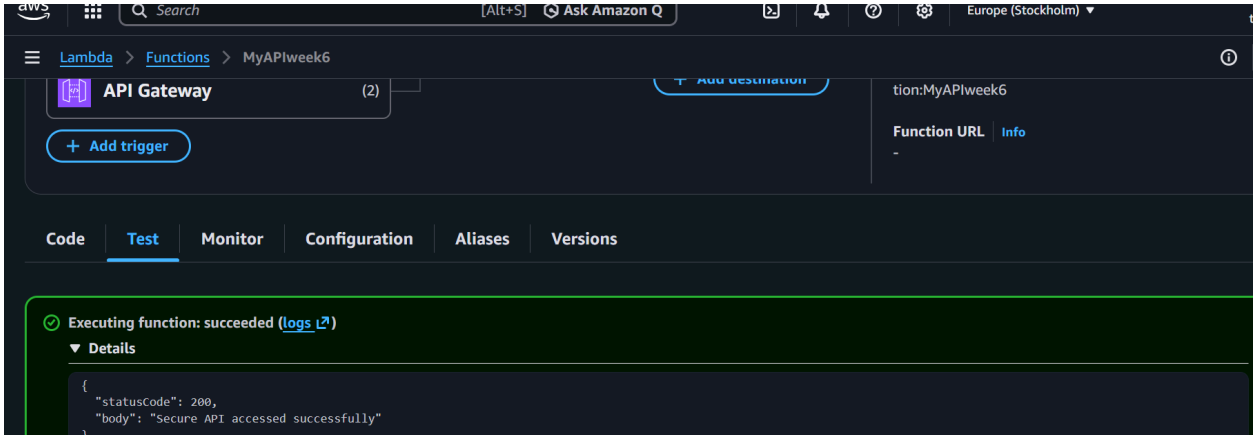
- Created and deployed a Lambda function returning a secure message
- Built a REST API in API Gateway and connected it to the Lambda

- Developed and configured a Lambda authorizer for token validation
- Applied throttling and rate limiting at the stage level
- Tested authentication and authorization with valid/invalid tokens

4. Implementation Details

4.1 Backend API (Lambda Function)

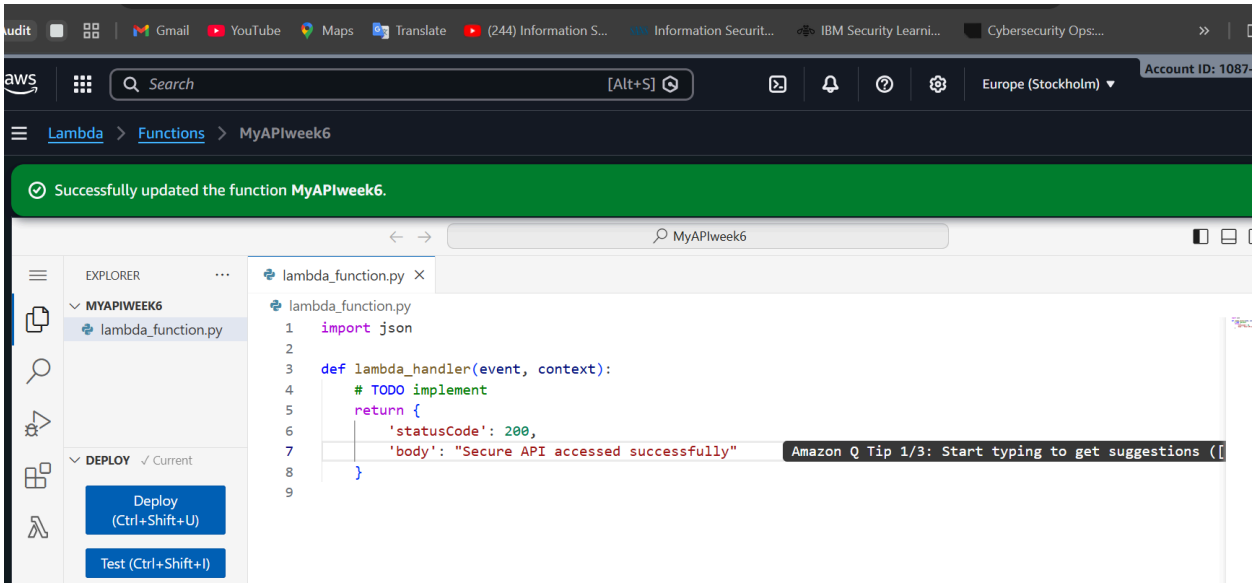
- **Function:** MyAPIweek6
- **Response:** `{ "statusCode": 200, "body": "Secure API accessed successfully" }`
- **Trigger:** API Gateway
- **Evidence:**



4.2 API Gateway Setup

- **API Name:** WEEK6API (later myWEEK6_api)
- **Resource Path:** /WEEK6
- **HTTP Methods:** GET, PUT
- **Deployment Stage:** week6

- **Evidence:**

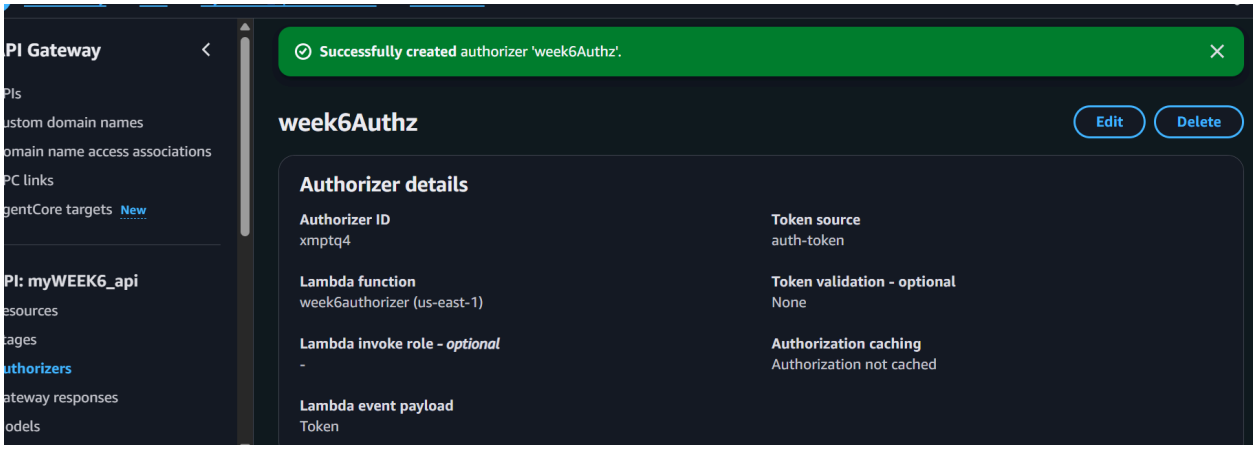


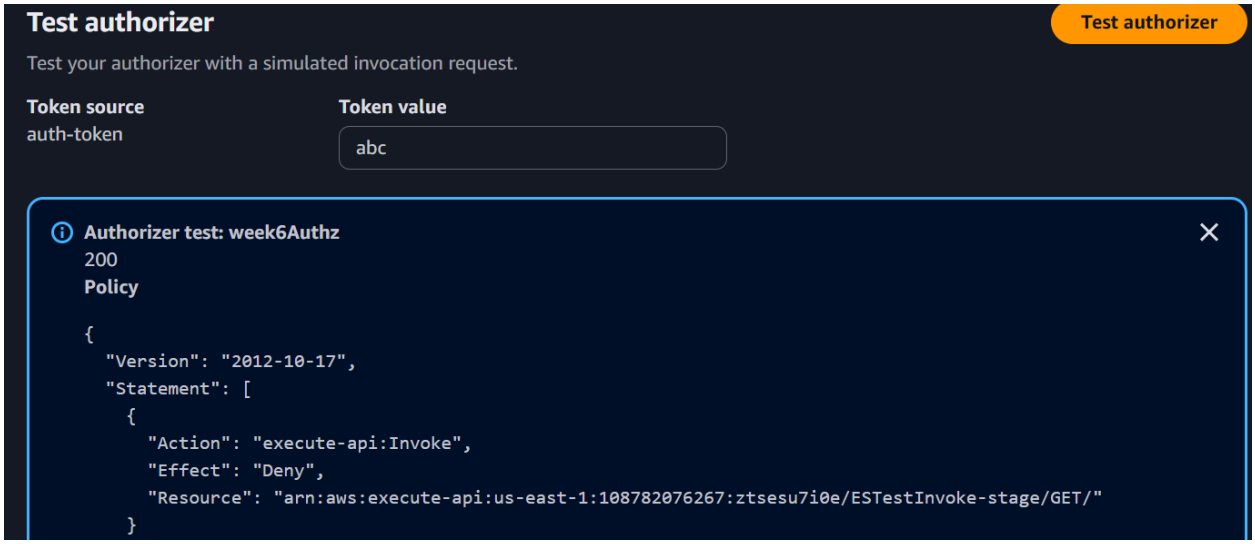
4.3 Authentication & Authorization

- **Authorizer:** week6Authz (Lambda-based)
- **Token Source:** Custom header `auth-token`

Validation Logic:

- Token `xyz987` → Allow
- Token `abc` → Deny
- **Policy:** Returns IAM-style Allow/Deny policies
- **Evidence:**





4.4 Security Controls Applied

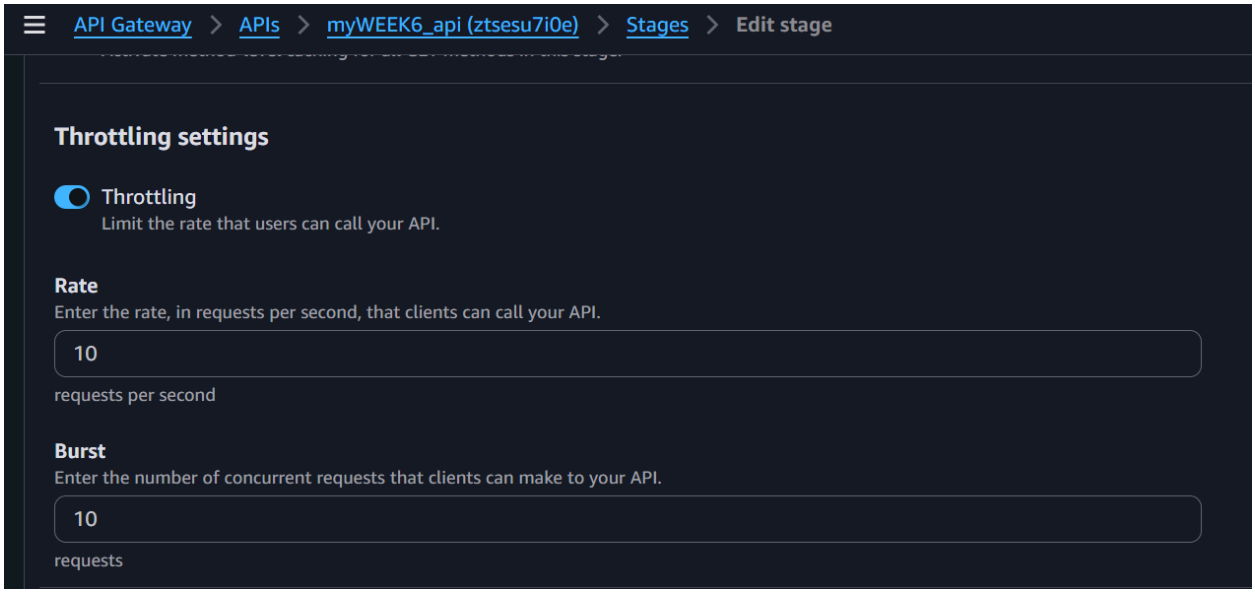
a) HTTPS Only

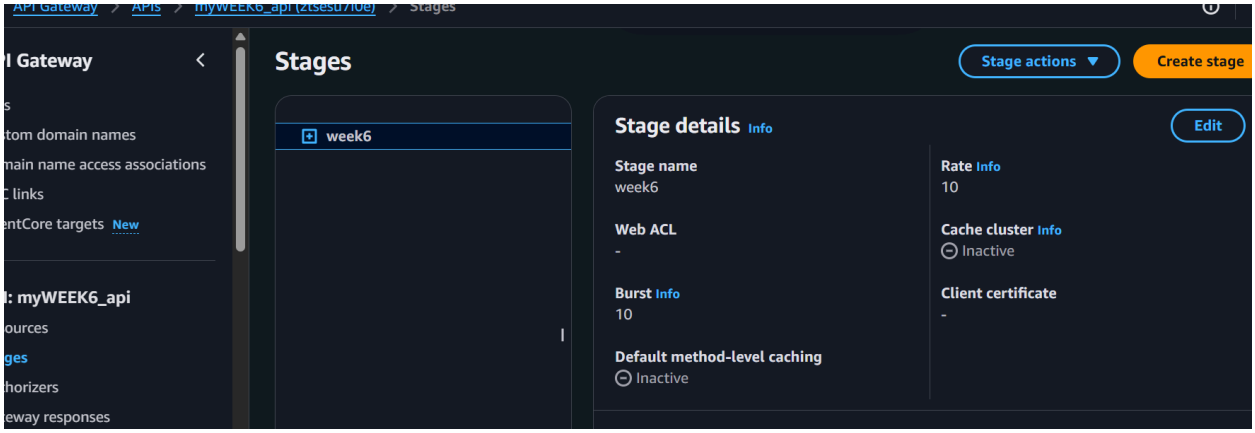
- All endpoints served via HTTPS by default
- Invocation URL: https://ztsesu7i0e.execute-api.us-east-1.amazonaws.com

b) Rate Limiting

- Rate: 10 requests per second
- Burst: 10 concurrent requests
- Configured at the API stage level

- **Evidence:**





c) Access Control

- Token-based authorization via Lambda authorizer
- Explicit Deny policy for invalid/missing tokens
- Resource-level authorization enforced

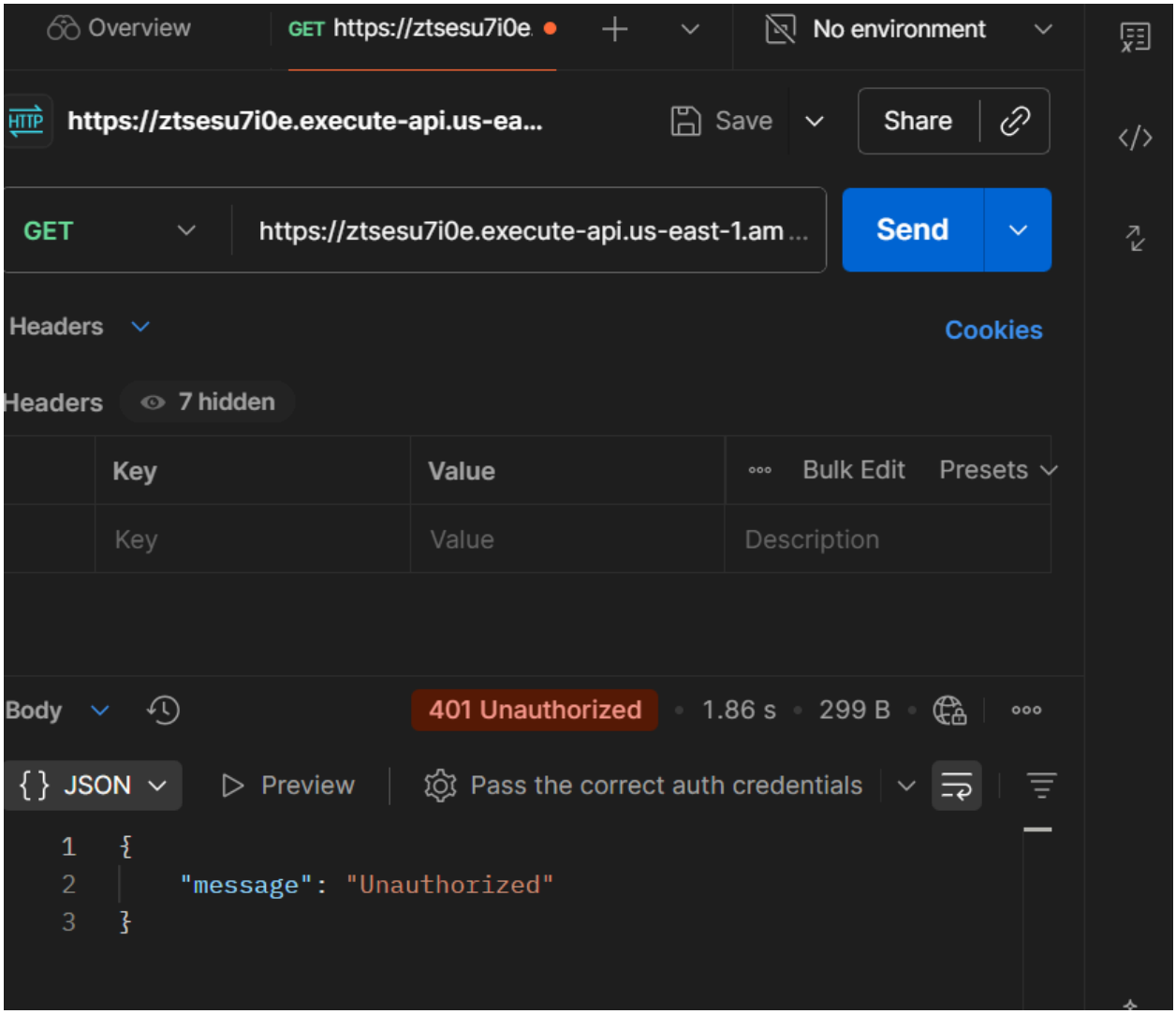
---

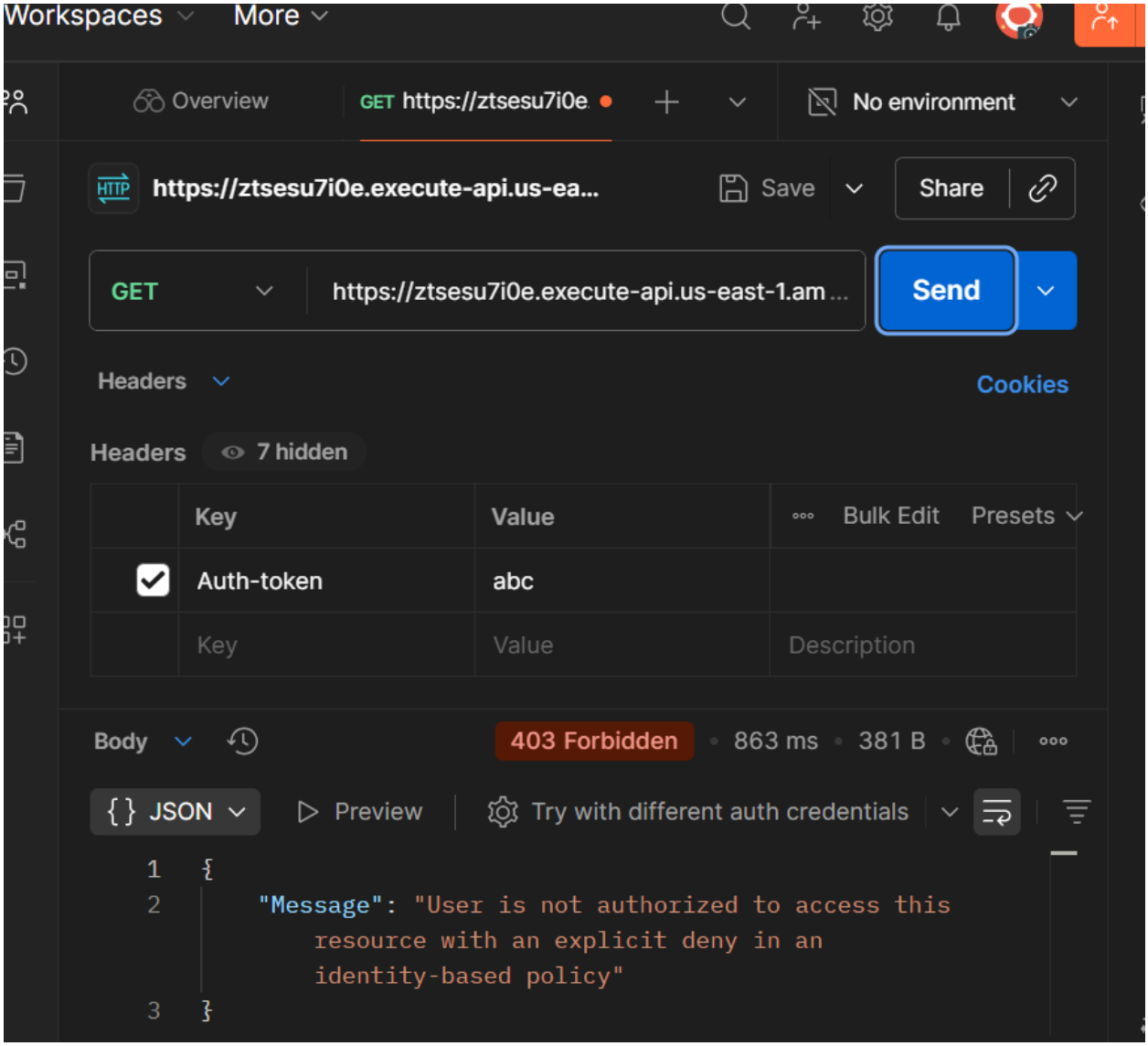
5. Verification & Testing

5.1 Unauthorized Access Attempt

- **Request:** Without token or with invalid token (abc)
- **Response:** 401 Unauthorized or 403 Forbidden
- **Evidence:**

-
ztsesu7i0e.execute-api.us-east-1.amazonaws.com/week6/API1
Audit
Gmail
YouTube
Maps
Translate
(244) Information S...
Information Securit...
IBM Security Learni...
Cybersecurity Ops:...
Pretty print
{"message":"Unauthorized"}

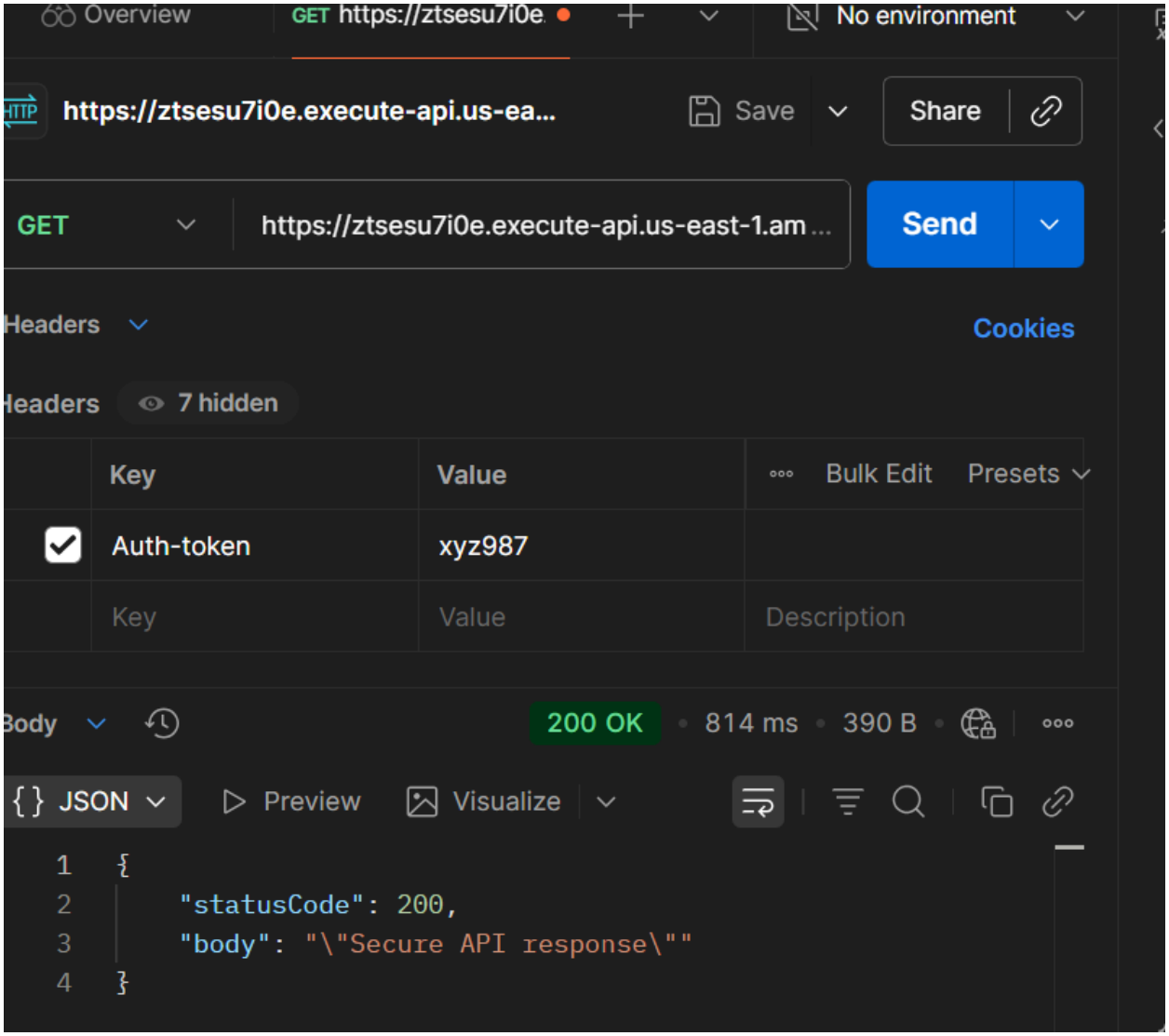

Overview
GET https://ztsesu7i0e
No environment
https://ztsesu7i0e.execute-api.us-ea...
Save
Share
GET
https://ztsesu7i0e.execute-api.us-east-1.am ...
Send
Headers
Cookies
Headers
7 hidden
Key
Value
Bulk Edit
Presets
Key
Value
Description
Body
401 Unauthorized
1.86 s
299 B
JSON
Preview
Pass the correct auth credentials
1 {
2 "message": "Unauthorized"
3 }



5.2 Authorized Access

- **Request:** With valid token (`xyz987`) in `auth-token` header
- **Response:** 200 OK with Lambda response body

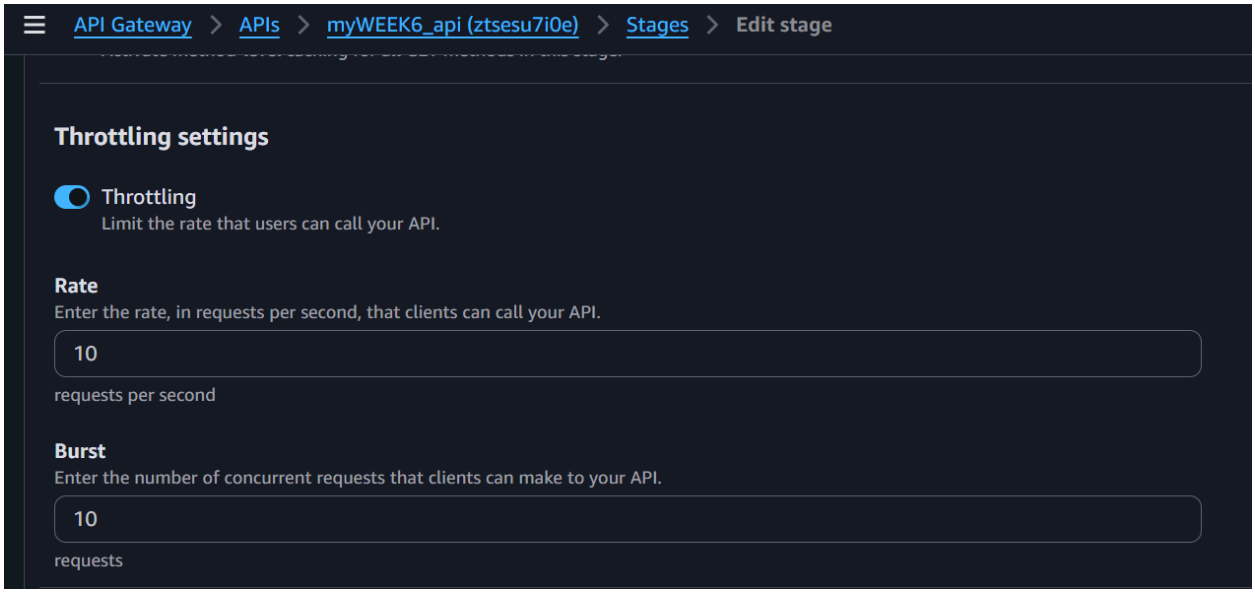
- **Evidence:**



5.3 Throttling Configuration

- Settings verified and applied in stage configuration

- **Evidence:**



---

6. Conclusion

The lab successfully demonstrated the implementation of a secure, serverless API on AWS using API Gateway and Lambda. By integrating a custom Lambda authorizer, the API enforces token-based authentication and granular authorization. Additional security controls—HTTPS enforcement, rate limiting, and explicit access policies—ensure the API is protected against common threats such as unauthorized access, credential leakage, and denial-of-service attempts.